Trabajo Práctico - Aprendizaje Automático I

Grupo "Pablo Brusco Galactico"
Mario Livia Auqui Mauricio Enrich Tomas Mele
Facundo Sanchez Ignacio Lucas Rodriguez
Facultad de Ciencias Exactas y Naturales de la Universidad de Buenos Aires

1. Introducción

La transición de estados de hiperplasia celular hacia lesiones tumorales malignas representa uno de los enigmas más complejos en la medicina. Con el propósito de mejorar la detección temprana y predecir la evolución médica de pacientes con antecedentes familiares y lesiones pre-tumorales, dos grupos de investigación del CONICET han integrado la secuenciación de nueva generación (NGS) con herramientas de inteligencia artificial.

Este trabajo práctico tiene como objetivo principal desarrollar y evaluar modelos predictivos de clasificación binaria capaces de identificar la evolución de estas lesiones basándose en perfiles de expresión genética. A partir de un estudio de 5 años de duración sobre un grupo de pacientes con antecedentes familiares, se buscará predecir si una muestra presenta un pronóstico favorable o si, por el contrario, existe riesgo de recaída.

2. Metodología

A partir del estudio mencionado, se extrajo nuestro conjunto de datos de desarrollo, compuesto por 500 instancias. Cada una contiene las mediciones de 200 genes ($X \in \mathbb{R}^{500\times200}$) y una etiqueta binaria (*target*) que indica un **buen pronóstico** (no hubo indicios de nuevas hiperplasias) o un **mal pronóstico** (hubo una recaída).

Debido a que el conjunto de datos presentaba un desbalance (la clase positiva es aproximadamente el $28\,\%$ de las muestras), cada partición del conjunto de datos se realizó de manera estratificada para mantener una misma proporción de clases. Si no tuvieramos esto en cuenta, nuestro modelo podría no tener suficientes datos de una clase para poder generalizar.

Durante la etapa de desarrollo, reservamos una porción de los datos ($20\,\%$) para que actue como un conjunto de control. Al realizar esta separacion, se hizo la suposición de que cada instancia era un paciente diferente, ya que si no, los modelos podrian estar siendo afectados por la correlación que habría en un mismo paciente y estar ajustando a esto y no a los patrones generales.

Con el $80\,\%$ restante de los datos, se procedió a la fase de desarrollo. Como primer modelo, se entrenó un árbol de decisión con una altura máxima de 3 y el resto de los hiperparámetros por defecto. Posteriormente, para evaluar el impacto de distintas configuraciones en este modelo inicial, se implementó una búsqueda exhaustiva sobre un espacio reducido de hiperparámetros.

Para garantizar una evaluación robusta a lo largo del resto del trabajo, se estableció el uso de validación cruzada estratificada de 5 particiones (K-Fold CV con $K = 5$) al entrenar nuevos modelos. Con este fin, se desarrolló una función personalizada (*stratifiedKFoldCV*) que no solo gestiona las particiones, sino que calcula diversas métricas por *fold*, sus promedios y el global (como se vió en clase) del modelo: *Accuracy*, AUC-PRC, AUC-ROC (estas 3 últimas en entrenamiento y validación), F1-Score y *Recall Precisión* (estas dos últimas segregada por clase). Teniendo que las particiones de datos y las probabilidades predichas son determinísticas, la función permite modificar el umbral de decisión (fijado en $0{,}5$ por defecto) y recalcular las métricas que dependen de dicho umbral, manteniendo inalterables las predicciones subyacentes y las particiones que se producen.

La búsqueda de las mejores configuraciones para los algoritmos de árboles de decisión, KNN y SVM se realizó buscando un subconjunto de manera aleato-

ria (`RandomizedSearchCV`) la cual utiliza la validación cruzada mencionada anteriormente. Como criterio a optimizar, se buscó maximizar AUC-ROC. Una vez identificadas las mejores configuraciones, estas fueron evaluadas nuevamente utilizando nuestra función `stratifiedKFoldCV` para obtener un reporte de métricas. Este mismo procedimiento de evaluación se aplicó a los modelos de Análisis Discriminante Lineal (LDA) y Naïve Bayes, utilizando sus hiperparámetros por defecto.

Por ultimo, se comparo el AUCROC de cada modelo, eligiendo al mejor. Luego, se calculó el umbral óptimo para cada mejor modelo para comparar métricas que dependan un umbral. El criterio que utilizamos fue "closes-to-(0,1)", intuitivamente, encuentra el umbral más cercano al modelo perfecto: cero falsos positivos y cero falsos negativos[1]. Finalmente, se predijeron las clases para reportar.

3. Hiperparámetros

En esta sección detallamos la configuración de los hiperparámetros explorados durante la fase de desarrollo de distintos modelos. Durante la exploración mediante búsqueda aleatoria (*Randomized Search*), se buscó evaluar entre el $20\,\%$ y el $25\,\%$ de las combinaciones posibles porque realizar una búsqueda exhaustiva (*Grid Search*) resultaría computacionalmente muy costoso dada la alta dimensionalidad de los datos (200 genes) y la complejidad de algoritmos como SVM. Además, explorar una fracción aleatoria del espacio de búsqueda es altamente eficiente para encontrar configuraciones muy cercanas a la óptima, dado que usualmente solo un subconjunto reducido de hiperparámetros son los çorrectos"para obtener la mejor performance del modelo. Los resultados obtenidos a partir de estas configuraciones se reportan posteriormente en la Apartado 4.

3.1. Árboles de Decisión

- **Max Depth**: Limita la profundidad máxima del árbol. Se exploraron los valores 3, 4, 5, 6, 7 y `None` (sin límite).
- **Min Samples Split**: Establece la cantidad mínima de muestras requeridas en un nodo para permitir una nueva división. Se evaluaron los valores $\{15, 20, 25, 30, 35, 40\}$. Valores menores prolongan el crecimiento del árbol arriesgando sobreajuste, mientras que valores mayores actúan como regularización forzando una detención temprana.
- **Max Features**: Determina la cantidad de atributos a considerar como candidatos al evaluar un corte. Se utilizaron las opciones: `None` (todos los atributos), `'sqrt'` y `'log2'`.
- **Criterion**: Define la función utilizada para medir la mejor partición en los nodos. Se exploraron las opciones `'gini'` y `'entropy'`.

Para el espacio de búsqueda para los Árboles de Decisión se consideró el riesgo de *overfitting* intrínseco de este modelo. Por lo tanto, la elección de los hiperparámetros se centró en explorar distintos mecanismos de regularización. Variamos la profundidad y exigimos un número de muestras para realizar divisiones con el fin de evitar que el árbol memorice ruido o casos atípicos. Asimismo, la inclusión de `'sqrt'` y `'log2'` en la selección de atributos busca mitigar la alta dimensionalidad, forzando al algoritmo a descubrir patrones en distintos subconjuntos de genes.

3.2. K-Nearest Neighbors (KNN)

- **K Neighbors**: Se evaluaron valores entre 1 y 20. Debido a que 30 vecinos representa el $5\,\%$ del conjunto de entrenamiento (400 instancias), considerar un k mayor se vería demasiado afectado por el desbalance de clases.
- **Metric**: la distancia euclídea sufre de la maldición de la dimensión, entonces, decidimos probar otras métricas. Se compararon las distancias `'cityblock'` o d1, `'euclidean'` (d2) y `'cosine'`, esta última es más robusta frente a la maldición .
- **Weights**: Define un peso en los votos de los vecinos. Usando `'uniform'` el cada voto pesa igual y con `'distance'` cada voto se multiplica por $\frac{1}{d(nodo,vecino)}$. Seleccionamos esté hiperparámetro para intentar balancear los votos de las clases, lamentablemente no hay ninguno que pese por la proporción de las clases.

3.3. Support Vector Machines (SVM)

- **Kernel**: Se evaluaron los kernels `'linear'`, `'poly'` y `'rbf'`, estos dos ultimos proyectan

[1] Es el punto (TPR,FRP) que minimiza d, con $d = \sqrt{(1 - TPR)^2 + FPR^2}$.

las instancias a dimensiones superiores para definir fronteras más complejas. Hiperparámetro necesario si la frontera real no es lineal o capturar interrelaciones entre atributos.

- **C** (Penalización común a todos los kernels): Controla la cantidad y gravedad de las violaciones del margen. Se utilizó un rango amplio en escala logarítmica dado su impacto no lineal en el rendimiento. Valores pequeños genera un margen limitado en ancho (riesgo de sobreajuste), mientras que valores altos permiten errores en el entrenamiento en favor de una mejor generalización, con un margen ancho.
- **Gamma** (Para RBF): Determina el radio de influencia de cada vector de soporte. Un valor alto hace que la distancia 'efectiva' entre vectores de soporte y nuevas instancias caiga exponencialmente más rápido. Por lo que un vector de soporte cercano va a tener mayor peso al momento de determinar el signo de la nueva instancia a partir del plano. Esto puede generar 'fronteras islas' y/o altamente complejas. Se exploró en escala logarítmica con umbrales máximos y mínimos reducidos para evitar casos de propensos a sobreajuste o subajuste. Necesario, ya que la varianza entre atributos es muy alta.
- **Degree** (Para Polinomial): Define el grado del polinomio y, por ende, la complejidad de las combinaciones de atributos, ya que se capturan las interralaciones entre los atributos. Se priorizaron grados bajos para limitar el costo computacional exponencial y porque el kernel RBF ya cubre la búsqueda de fronteras de alta complejidad.
- **Coef0** (Para Polinomial): Controla la influencia de los términos independientes y de grados inferiores en la función polinómica. Se utilizaron los valores por defecto ante la falta de conocimiento a priori sobre las relaciones intrínsecas de los atributos.

3.4. Modelos Generativos

Adicionalmente, para los algoritmos generativos identificamos hiperparámetros que no fueron incluidos en la experimentación dado que utilizamos sus valores por defecto, pero cuya exploración consideramos relevante. Las implicancias de estos parámetros no utilizados se abordan en la Apartado 5.

Análisis Discriminante Lineal (LDA)

- **Shrinkage**: Introduce una regularización que "traslada"la matriz de covarianzas estimada hacia una matriz diagonal, asumiendo mayor independencia entre los atributos. En este contexto de alta dimensionalidad, ayuda a suprimir correlaciones espurias producto del ruido muestral, mejorando la estabilidad del modelo. Sin embargo, un encogimiento excesivo podría difuminar correlaciones reales y sutiles que son necesarias para distinguir correctamente la clase minoritaria.
- **Solver**: Define el algoritmo interno que se utilizará para calcular las fronteras de decisión y encontrar los componentes de maximización de separación entre clases. Su valor por defecto es 'svd', que configura al modelo a buscar la solución mediante la descomposición en valores singulares, pero no es compatible con shrinkage. Por esto, deberían explorarse 'lsqr' o 'eigen', para evaluar si es mas conveniente utilizar mínimos cuadrados o descomposición de autovalores.

Naive Bayes

- **Priors**: Permite forzar las probabilidades previas de cada clase. Especificar probabilidades previas balanceadas (por ej. $50/50$) obligaría al modelo a basar su decisión puramente en la verosimilitud (distribución de los atributos), aunque a costa de ignorar la proporción real.
- **Var Smoothing**: Actúa como un mecanismo de regularización al agregar varianza artificialmente. Al incrementar este valor, se ënsanchanl̈as distribuciones, suavizando las fronteras de decisión y previniendo el sobreajuste frente a pequeñas variaciones ruidosas en el conjunto de entrenamiento. La contrapartida es la introducción de un sesgo intencional que aplana las verdaderas distribuciones de los datos.

4. Resultados

4.1. Árboles de decision.

Como punto de partida, se entrenó un árbol de decisión con una profundidad máxima de 3 sobre el conjunto de desarrollo, usando los hiperparáme-

tros por defecto. Los datos de desarrollo fueron nuevamente divididos en conjuntos de entrenamiento y validación mediante la función implementada `train_test_split` realizando así validación cruzada, y se calculó la métrica de *accuracy* en ambos subconjuntos. Se obtuvo un score de $0{,}848$ en los datos de desarrollo, frente a un $0{,}646$ en los datos de validación.

El resultado obtenido en validación es, incluso, peor que el que daría un modelo *dummy* que prediga siempre la clase mayoritaria, el cual alcanzaría un desempeño cercano al $0{,}72$ en *accuracy*.

Para evaluar el modelo aún más, se aplicó validación cruzada utilizando *Stratified K-Fold* con K = 5 y el umbral default de $0{,}5$. En cada iteración se calcularon las métricas de *accuracy*, AUC-PRC y AUC-ROC, registrando también los promedios y el rendimiento global tanto para el conjunto de desarrollo como para el de validación. Los resultados de este análisis se detallan a continuación en la Tabla 1:

	Accuracy		**AUC-PRC**		**AUC-ROC**	
Perm.	**Tr**	**Val**	**Tr**	**Val**	**Tr**	**Val**
0	0.825	0.728	0.689	0.458	0.844	0.721
1	0.841	0.613	0.692	0.353	0.864	0.619
2	0.813	0.688	0.658	0.420	0.846	0.737
3	0.829	0.650	0.706	0.412	0.871	0.671
4	0.832	0.613	0.682	0.372	0.849	0.673
Prom.	0.828	0.658	0.685	0.403	0.855	0.684
Global	NO	0.658	NO	0.395	NO	0.683

Tabla 1: Performance del Árbol de altura 3 con K-Fold CV ($K = 5$). Tr: Train, Val: Validación.

En la Tabla 1 se puede observar que tanto para el global, como para cada valor particular de las distintas permutaciones, se da una diferencia significativa entre AUC-PRC y AUC-ROC, siendo este último similar al *accuracy*. Interpretamos este resultado como un indicio de que el árbol está aprendiendo a excluir la clase negativa, sin darle demasiada importancia a la clase positiva, pues hay muchas menos muestras con las que entrenar. Como consecuencia, muchas instancias de la clase positiva son clasificadas como negativas.

Posteriormente, se exploraron las configuraciones listadas en la Tabla 2. Los resultados obtenidos indican que incrementar la profundidad del árbol no tiene diferencia practicamente. Esto sugiere que, para este problema en particular, los arboles no son un bueno modelo.

Criterio	**Max Depth**	**Accuracy (Train)**	**Accuracy (Validación)**
gini	3	0.828	0.658
gini	5	0.933	0.683
gini	∞	1.00	0.686
entropy	3	0.804	0.678
entropy	5	0.886	0.656
entropy	∞	1.00	0.658

Tabla 2: Performance de Árboles de Decisión con diferentes parámetros.

Por último, se llevó a cabo una exploración de los hiperparámetros listados en el Apartado 3. Para esto, se usó *RandomizedSearchCV* con 50 iteraciones, usando AUC-ROC como métrica de performance y 5 folds para validación cruzada. Las mejores 5 configuraciones se pueden ver en la Tabla 3. Esto nos muestra que si bien existen configuraciones óptimas para este algoritmo, el AUC-ROC no logra superar el umbral de 0.69. Esto puede atribuirse a una combinación entre el sesgo inductivo del algoritmo y la alta complejidad en la frontera de decisión de los datos que no probablemente esten lejos de ser hiperrectángulos en R^{200}.

Rank	**AUC-ROC**	**Min Split**	**Depth**	**Criterio**
1	0.690 ± 0.040	15	4	gini
2	0.678 ± 0.028	25	6	gini
3	0.677 ± 0.031	35	7	gini
4	0.671 ± 0.057	40	3	gini
5	0.659 ± 0.061	20	5	gini

Tabla 3: Resultados Árboles de Decisión (Media $\pm$ Std). En todas las configuraciones ganadoras el parámetro *Max Features* fue `None`.

4.2. KNN

Las configuraciones exploradas junto con la mejor configuración se encuentran resumidas en la Tabla 4. Los resultados de la mejor configuración para KNN con el umbral default de $0{,}5$, se encuentran en la Tabla 6. Podemos ver que los mejores modelos pesan a los vecinos según la distancia *coseno* que posee mayor robustez en dimensiones altas que la euclídea. Esto se debe a que en espacios de alta dimension, para algunas distancias, como la euclídea, los puntos

se encuentran ïgual de lejosz el concepto de cercanía pierde sentido. Este fenómeno es conocido como la maldición de la dimensionalidad. En cambio, para la distancia coseno solo interesa la diferencia entre el ángulo de los vectores que representan la instancia y no su magnitud.

También se puede observar que los mejores modelos usan muchos vecinos para maximizar el score. Creemos que este algoritmo no mejora aún más ya que no es robusto cuando hay desbalance de clases, pues no pondera las proporciones. Luego al aumentar el número de vecinos, naturalmente, clasificará a los datos en gran parte como la clase mayoritaria obteniendo un resultado parecido a la proporción de la clase mayoritaria.

Rank	AUC-ROC	Pesos	K	Métrica
1	0.852 ± 0.045	distance	17	cosine
2	0.850 ± 0.051	distance	19	cosine
3	0.848 ± 0.036	distance	14	cosine
4	0.845 ± 0.040	distance	11	cosine
5	0.844 ± 0.043	uniform	18	cityblock

Tabla 4: Resultados de la búsqueda para KNN (Media $\pm$ Std).

4.3. SVM

Las configuraciones exploradas junto con la mejor configuración se encuentran resumidas en la Tabla 5. La mejor configuración tiene kernel RBF, gamma scale = 0.00026 y C igual a 100. Observamos que el resto de los mejores modelos (hasta el top 14) son kernels polinomiales, por lo que intuimos que hubo un parámetro decisivo para RBF: gamma. [2] El gamma pequeño se debe a que los datos tienen una varianza muy grande (sin distinguir entre features). Esto hace que la influencia (peso) del vector de soporte también sea muy grande. Se obtuvo un AUC-ROC promedio de 0.881 y global de 0.880 durante desarrollo, esto lo hace el mejor modelo según la métrica que elegimos.

[2] $gamma = \frac{1}{n \cdot \mathrm{Var}(X)}$, donde n es la cantidad de features.

Rank	AUC-ROC	Kernel	Degree	Coef0	C	Gamma
1	0.881 ± 0.059	rbf	-	-	100.0	scale
2	0.867 ± 0.066	poly	2.0	0.1	10.0	-
3	0.867 ± 0.067	poly	2.0	0.1	1000.0	-
4	0.867 ± 0.054	poly	3.0	0.5	1.0	-
5	0.862 ± 0.064	poly	2.0	0.5	10.0	-

Tabla 5: Resultados de la búsqueda para SVM (Media $\pm$ Std). Se utiliza "para denotar parámetros que no aplican al kernel evaluado.

4.4. Modelos Generativos

En cuanto a modelos generativos, se decidió evaluar *LinearDiscriminantAnalysis* y Naïve Bayes con sus parámetros por defecto. En cuanto a LDA, los resultados se pueden observar en la Tabla 6 (su AUC-ROC y demás métricas con umbral óptimo). Una posible explicación de estos resultados es que LDA se ve fuertemente limitado por la demiension de los datos en relación a la cantidad de instancias (200 atributos para 400 instancias), dicha proporción dificulta el cálculo interno para estimar la matriz de covarianzas, volviéndola inestable.

Por otro lado, se evaluó Gaussian Naïve Bayes. Los resultados en la Tabla 6 (su AUC-ROC y demás métricas con umbral óptimo) son inesperadamente bajos. Una posible explicación para este desempeño moderado es que, si bien hay un cierto grado de independencia condicional, esta no es total.

4.5. Mejor Modelo

Para resumir todos los experimentos abarcados, se generó una tabla que resume algunas métricas de los informes de cada modelo con su mejor configuración ordenadas por su resultado con AUC-ROC.

Aquí podemos apreciar cómo SVM es el modelo superior, obteniendo el mayor score de AUC-ROC. Comparando otras métricas, dados los umbrales óptimos, SVM tiene un F1 superior por diferencia, que se debe a que tiene un recall y precisión más alto para ambas clases que los otros modelos (solo igualado en accuracy de la clase minoritaria [3]).

[3] Accuracy de la clase minoritaria, es decir la clase positiva, es equivalente al Recall.

Modelo	Umb.	Acc.	A.May	A.Min	F1	PRC	ROC
SVC	0.303	0.843	0.854	0.815	0.749	0.782	0.881
KNN	0.247	0.783	0.771	0.814	0.683	0.713	0.852
NB	0.097	0.663	0.670	0.644	0.520	0.555	0.729
D.Tree	0.179	0.661	0.660	0.665	0.522	0.407	0.690
LDA	0.026	0.581	0.559	0.636	0.460	0.436	0.626

Tabla 6: *Resultados promedio de validación cruzada ($K = 5$). Se muestra el umbral óptimo empírico (Umb.), accuracy general (Acc.), segmentado en clase mayoritaria (A.May) y minoritaria (A.Min), F1-Score (F1), AUC-PRC (PRC) y AUC-ROC (ROC).*

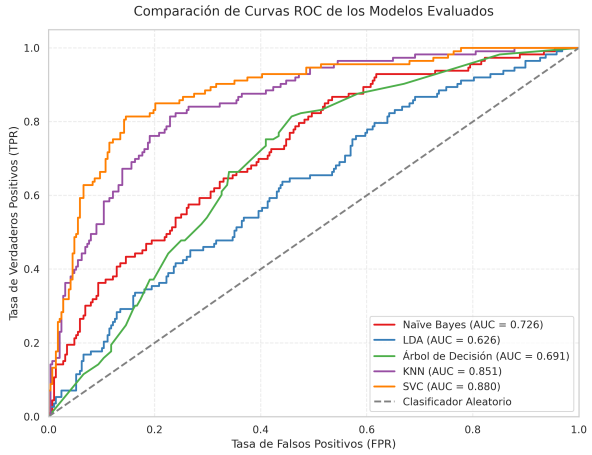


Figura 1: Comparación de las curvas ROC para los modelos evaluados en validación. Se destaca la superioridad del modelo SVM (AUC = 0.881) frente al resto de los clasificadores.

A partir de estos resultados, es posible analizar por qué SVM obtuvo la mejor performance considerando el comportamiento y las características de cada algoritmo.

Teniendo en cuenta las mejores configuraciones de SVM y KNN, creemos que el sesgo de 'clasificar una instancias con la clase de sus vecinos' es correcto. El kernel 'rbf' de SVM tiene en cuenta la distancia euclídea , mientras que en la mejor configuración de KNN se utiliza la distancia coseno, la cual ayuda con el problema de la dimension, pero es perjudicado por el desbalance de clases. ¿Cómo SVM soluciona la maldición? El kernel amplifica las distancias exponencialmente, cómo se ve en B. Creemos que el efecto es el siguiente: supongamos que el vector de soporte más cercano para la instancia x^i es de la clase positiva, entonces va a pesar exponencialmente más que el resto de los vectores, lo que solventa el desbalance de clases[4].

4.6. Reporte Control

Con el objetivo de estimar el valor de AUC-ROC que obtendremos al evaluar nuestro modelo con datos nuevos, utilizamos el conjunto que habíamos separado previamente en el Apartado 2. Evaluamos estas instancias empleando nuestro mejor modelo (SVM) y calculamos el umbral óptimo, la exactitud (*accuracy*) general y por clase, la precisión para las clases mayoritaria y minoritaria, el F1-Score, el AUC-PRC y el AUC-ROC (ver Tabla 7).

En comparación con la etapa de validación, el desempeño sobre este conjunto reservado se mantuvo muy alineado con los promedios reportados en la Tabla 6. Dada esta estabilidad, estimamos que el AUC-ROC final de nuestro modelo frente a los datos de control será de 0,8833.

5. Discusión

A lo largo de esta investigación, surgieron diversos tópicos e hipótesis que consideramos relevantes. Principalmente relacionados con problemáticas y preguntas que quedaron abiertas durante el desarrollo del trabajo. Por diferentes motivos, estas cuestiones no fueron abordadas en profundidad, ya sea porque el esfuerzo requerido se alejaba del objetivo principal de la investigación o por las características y limitaciones propias de los datos disponibles. A continuación, se presentan y desarrollan individualmente las principales cuestiones que surgieron durante el proceso.

5.1. Caracterización de los atributos

Creemos importante entender qué significan los valores de las mediciones con las que trabajamos, para explicar qué es lo que deben aprender los modelos de las mediciones y por qué algunos son mejores que otros en esto. Consideramos, luego de una investigación (elemental) estas son las características que consideramos más relevantes:

- Normalización de los datos: los datos están normalizados, pero no sabemos de qué forma exactamente. De acuerdo a la normalización utilizada tenemos que si el valor del atributo es

[4] A pesar de la gran cantidad de negativos, van a pesar mucho menos.

Modelo	Umbral Ópt.	Accuracy	Acc. May.	Acc. Min.	Prec. May.	Prec. Min.	F1-Score	AUC-PRC	AUC-ROC
Mejor SVM	0.303	0.848	0.901	0.714	0.889	0.741	0.727	0.778	0.883

Tabla 7: *Resultados de la evaluación del modelo final (Mejor SVM) sobre el conjunto de control. Se reportan las métricas globales y segmentadas por clase aplicando el umbral óptimo empírico.*

mayor a 0 significa que la expresión del gen es mayor que la media, en algún sentido. Para ejemplificar, supongamos que la medicón es 2.0, entonces la medición es 2.0 veces mayor que la media o 2.0 desviaciones estándar mayor que la media.

- Expresión de un gen.

¿Qué es la expresión de un gen? Una explicación simplificada es que los genes generan ARNs específicos, y la expresión es la medición de la cantidad de veces que aparece cierto ARN de un gen específico en una muestra. La evolución a células cancerosas se debe a una acumulación de mutaciones de ciertos genes y las mediciones de la expresión no implican que hayan ocurrido, ya que los cambios de expresión pueden deberse, también, regulación epigenética[5], etc. Además, hay mutaciones que no generan modificaciones en la expresión. Esto nos lleva a creer que los modelos deben hacer un trabajo más complejo del que pensabamos inicialmente, ya que no solo importan que puedan distinguir que genes son los más importantes, sino cuales son los más confiables dada su expresión. Esto es porque un gen podría ser determinante para el desarrollo de células cancerosas (cómo los que generan la muerte celular) pero las mediciones de su expresión no permiten detectar mutaciones. Escapa de nuestro conocimiento e investigación sobre el tema una forma de selección efectiva (no heurísticas).

5.2. No linearidad en la frontera de decisión

Una de las hipótesis que surgieron durante la elaboración de este trabajo es que la frontera de decisión de los datos podría presentar una naturaleza no lineal. Esto podría explicar las dificultades observadas en modelos como LDA, árboles de decisión y SVM Lineal para clasificar correctamente los datos, dado que estos imponen ciertas restricciones sobre la forma de la frontera de decisión. En contraste, los mejores rendimientos fueron obtenidos mediante modelos como KNN y SVM con kernels gaussiano y polinómico. KNN no asume una forma paramétrica específica para la frontera, mientras que los kernels utilizados por SVM permiten representar fronteras de decisión no lineales mediante una transformación implícita del espacio de atributos. No obstante, consideramos que la corroboración de esta hipótesis requeriría un análisis específico de la estructura de los datos y de sus fronteras de decisión.

5.3. Transformación de los datos

Creemos que nuestros modelos podrían haberse beneficiado considerablemente de la incorporación de alguna técnica de transformación de los datos, particularmente de métodos de reducción de dimensionalidad, dado que las variables ya se encontraban escaladas. En la etapa de desarrollo contábamos con aproximadamente 200 atributos para tan solo 400 instancias, lo que representa una relación elevada entre dimensionalidad y cantidad de observaciones. En este contexto, una posible alternativa habría sido utilizar un método de selección de atributos que permitiera identificar aquellos con mayor capacidad para discriminar entre las clases y reducir así la complejidad del espacio de características como se realizó en el Apartado A. Esta situación podría haber afectado especialmente a LDA, ya que la estimación de su matriz de covarianzas puede verse comprometida cuando la cantidad de atributos es elevada en relación con el número de observaciones.

Al igual que LDA, QDA podría verse beneficiado enormemente. Anteriormente dada la alta dimensionalidad no era posible estimar su matriz de covarianza, si se viese reducida su dimensionalidad entonces se podría emplear este algoritmo. Además, como observamos en el Apartado A.1, reduciendo su dimensionalidad con los métodos allí presentados se tiende a obtener normales. Esto cumpliría los sesgos inductivos de QDA apoyandonos también en la no linealidad de los datos.

Otra alternativa a considerar habría sido la aplica-

[5] Es decir, qué partes del gen lee una célula o no.

ción de PCA, con el objetivo de proyectar los datos sobre un espacio de menor dimensionalidad conservando la mayor parte posible de su variabilidad.

6. Conclusiones

Luego de desarrollar, explorar hiperparámetros y comparar distintos algoritmos, el modelo de Support Vector Machines (SVM) con kernel radial (RBF) demostró ser el más apto para resolver este problema predictivo, alcanzando el mayor AUC-ROC en la fase de desarrollo.

Este resultado es esperado dados nuestros datos, con una alta dimensionalidad de 200 genes frente a una cantidad limitada de datos. Los árboles de decisión o modelos lineales como LDA se vieron superados por la aparente no linealidad de la frontera de decisión o por la incapacidad de poder estimar matrices de covarianza estables en un espacio tan grande. En contraste, algoritmos como SVM y KNN lograron mayor performance, siendo SVM el superior gracias a la capacidad del kernel RBF de proyectar los datos a dimensiones superiores.

En cuanto al problema biológico presentado, nuestros resultados sugieren que efectivamente existen patrones en la expresión genética del ARN que permiten predecir, en un buen porcentaje de casos, si una hiperplasia celular mantendrá un pronóstico favorable o evolucionará hacia una recaída maligna.

References

Pablo Brusco. Clases galácticas. In *Aprendizaje Automático*, 2C 2026.

Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, and Jonathan Taylor. *An Introduction to Statistical Learning: With Applications in Python*. Springer, 2 edition, 2023.

Nature Education. Gene expression is analyzed by tracking rna. Scitable by Nature Education, 2014.

Estos resultados del apéndice son un extra que decidimos sumar. Nos pareció copado reportarlos porque y creemos que le suman valor al trabajo.

A. Análisis de Datos

En esta sección analizamos la distribución subyacente de los atributos, un paso fundamental para justificar la elección de los algoritmos y comprender su desempeño.

A.1. Análisis de Normalidad

A.1.1. Normalidad Individual

A nivel univariable, el test de Shapiro-Wilks reveló que la inmensa mayoría de los 200 genes presenta una distribución normal condicionada por la clase (Tabla 8). Como ilustra la Figura 2, los comportamientos varían desde la normalidad global hasta la normalidad condicional asimétrica, existiendo un único gen que rechaza la distribución normal en ambas clases.

Clase	Cant. Genes Normales
0	188
1	185

Tabla 8: Cantidad de atributos que superaron la prueba de normalidad, separados por clase.

A.1.2. Normalidad Multivariable

Sin embargo, el escenario cambia drásticamente al evaluar el conjunto. El test de Henze-Zirkler sobre el conjunto de desarrollo rechazó el supuesto de normalidad multivariable para ambas clases al considerar los 200 atributos originales (Tabla 9).

Datos de Desarrollo	Norm. Multivariable	Valor p
Clase 0	Falso (Rechazada)	0
Clase 1	Falso (Rechazada)	0

Tabla 9: Resultados del test de Henze-Zirkler evaluado sobre el conjunto de desarrollo. Se rechaza la hipótesis nula de normalidad multivariable para ambas clases.

Para sortear este obstáculo, aplicamos reducción de dimensionalidad. Una primera selección heurística del *Top 40* de atributos con mayor diferencia de medias logró hacer emerger la normalidad multivariable, pero únicamente para la Clase 1 (Tabla 10).

Selección: Top 40 por Varianza de Medias		
Atributo Evaluado	**Resultado**	**Valor p**
Normalidad - Clase 0	Falso (Rechazada)	$< 0{,}001$
Normalidad - Clase 1	Verdadero (Aceptada)	$0{,}280$
Igualdad Covarianzas (Box's M)	Falso (Rechazada)	$< 0{,}001$

Tabla 10: Resultados multivariables para los 40 atributos con mayor varianza entre medias. Se observa una normalidad condicional asimétrica (solo presente en la Clase 1) y covarianzas heterogéneas.

Buscando una selección más limpia, optimizamos el criterio utilizando el Análisis de Varianza (ANOVA), el cual penaliza la varianza interna de las distribuciones para evitar el solapamiento de las clases. Al extraer el *Top 40* bajo este enfoque, logramos finalmente que el supuesto de normalidad multivariable se cumpla para ambas clases simultáneamente. A pesar de este éxito en las distribuciones conjuntas, el cálculo del test M de Box sobre estos subespacios reducidos dictaminó en todos los casos que las matrices de covarianza de ambas clases son significativamente heterogéneas (Tabla 11).

Selección: Top 40 mediante ANOVA		
Atributo Evaluado	**Resultado**	**Valor p**
Normalidad - Clase 0	Verdadero (Aceptada)	$0{,}387$
Normalidad - Clase 1	Verdadero (Aceptada)	$0{,}185$
Igualdad Covarianzas (Box's M)	Falso (Rechazada)	$< 0{,}001$

Tabla 11: Resultados multivariables para los 40 atributos seleccionados vía ANOVA. El uso de este criterio robusto devela un subespacio latente donde ambas clases se distribuyen como gaussianas multivariables, aunque manteniendo matrices de covarianza disímiles.

A.2. Análisis de Correlación condicionado por Clase

Para evaluar la multicolinealidad, calculamos la correlación de Spearman entre todos los pares de genes condicionados por clase. Dado que ningún par superó un coeficiente absoluto de $0{,}70$, descartamos la existencia de correlaciones fuertes que puedan perjudicar críticamente el desempeño de los modelos.

Asimismo, la Figura 3 respalda esta tendencia a la independencia condicional al evidenciar una fuerte concentración de los valores cerca del cero. No obstante, notamos una clara diferencia entre clases.

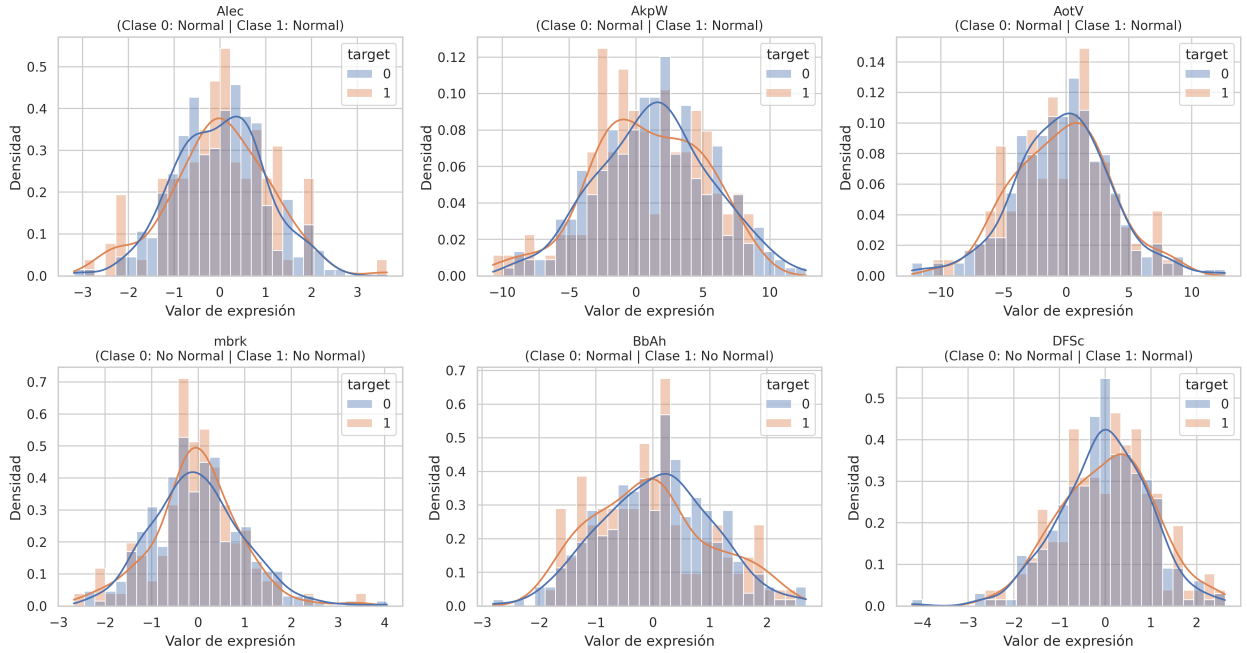


Figura 2: Comparación de densidades condicionadas a la clase. Se observa cómo la asunción de normalidad puede cumplirse globalmente (fila superior), ser rechazada en su totalidad, o depender estrictamente de la clase analizada (fila inferior).

La Clase 0 exhibe una independencia mucho más concentrada, mientras que la Clase 1 presenta una mayor dispersión, lo que indica que sus atributos están ligeramente más interrelacionados.

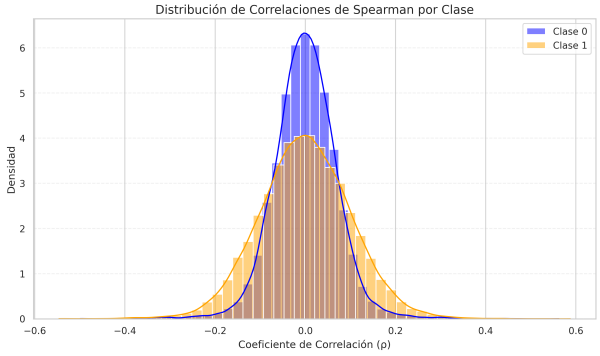


Figura 3: Distribución de los coeficientes de correlación de Spearman para todos los pares de genes, condicionado por clase.

B. Kernel RBF: Breve explicación

Definición:

$$K(x_i, x_{i'}) = \exp\left(-\gamma \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2\right).$$

Interpretación: Sea la observación de prueba

$$x^* = (x_1^*, \ldots, x_p^*)^T.$$

Si está lejos de una observación de entrenamiento x_i en términos de la distancia euclídea, entonces

$$\sum_{j=1}^{p} (x_j^* - x_{ij})^2$$

será grande, y por lo tanto

$$K(x^*, x_i) = \exp\left(-\gamma \sum_{j=1}^{p} (x_j^* - x_{ij})^2\right)$$

será muy pequeño (inversa de una exponencial). Esto significa que en x_i prácticamente no tendrá ningún papel en $f(x^*)$. Recordemos que la etiqueta de clase predicha para la observación de prueba x^* se basa en el signo de $f(x^*)$. En otras palabras, las observaciones de entrenamiento que están lejos de x^* no desempeñarán esencialmente ningún papel en la etiqueta de clase predicha para x^*. Esto significa que el kernel radial tiene un comportamiento muy local, en el sentido de que solo los vectores de soporte cercanos tienen efecto sobre la etiqueta de clase de una observación de prueba.